

Тема урока: методы экземпляра класса

1. Методы экземпляра класса
2. Метод `__init__()`
3. Аргумент `self`

Аннотация. Урок посвящен методам экземпляров классов в Python.

Методы экземпляра класса

Если атрибуты отображают некоторые характеристики, которые свойственны объектам определенного класса, то методы определяют их поведение. В Python существует несколько типов методов, но в этом уроке мы сосредоточимся только на **методах экземпляра класса**. Метод экземпляра — это функция, которая определена внутри класса, принадлежит объекту этого класса и имеет доступ к атрибутам объекта.



Как правило, названия методов экземпляра являются глаголами и соответствуют соглашению Snake case.

Рассмотрим пустой класс `Cat`, определим внутри него методы экземпляра `say()` и `eat()` и попробуем применить их к конкретному объекту.

Приведенный ниже код:

```
class Cat:
    def say(self):
        print('Мяу')

    def eat(self):
        print('Мням')

cat = Cat()

cat.say()
cat.eat()
```

ВЫВОДИТ:

Мяу
Мням

Обратите внимание, что за исключением того, что эти определения методов появляются в классе, они похожи на любое другое определение функции в Python. Они начинаются со строки заголовка, после которой идет выделенный отступом блок кода. Также следует заметить, что каждый метод имеет параметр `self`.

Параметр `self` необходимо указывать в каждом методе экземпляра. Когда метод вызывается, параметр `self` в качестве аргумента принимает тот экземпляр, через который этот метод был вызван. Именно этим экземпляром метод и будет оперировать.



Параметр `self` часто называют **контекстным объектом**.

Иными словами, привычные нам вызовы методов через точечную нотацию:

```
class Cat:
    def say(self):
        print('Мяу')

    def eat(self):
        print('Мням')

cat = Cat()

cat.say()
cat.eat()
```

Python преобразует в следующее:

```
class Cat:
    def say(self):
        print('Мяу')

    def eat(self):
        print('Мням')

cat = Cat()

Cat.say(cat)
Cat.eat(cat)
```

поэтому оба приведенных выше кода являются эквивалентными и выводят:

Мяу
Мням

Методы экземпляра доступны всем экземплярам соответствующего класса, поэтому мы можем создать несколько объектов класса `Cat`, и все они будут иметь общий функционал.

Приведенный ниже код:

```
class Cat:
    def say(self):
        print('Мяу')

    def eat(self):
        print('Мням')

cat1 = Cat()
cat2 = Cat()

cat1.say()
cat2.eat()
```

ВЫВОДИТ:

```
Мяу
Мням
```

Важно отметить, что параметр `self` — это обычная переменная, она может называться по-другому, но так категорически не рекомендуется делать. Соглашение об имени контекстного объекта — самое строгое из всех соглашений в мире Python. Если нарушить это соглашение, другие программисты просто не будут понимать ваш код. Кроме того, некоторые текстовые редакторы подсвечивают слово `self` определенным цветом, что довольно удобно.

Таким образом, при создании собственных методов экземпляра следует помнить о двух моментах:

- метод должен быть определен внутри класса (добавляется уровень отступов)
- метод всегда должен иметь хотя бы один параметр, и первый по счету параметр должен называться `self`

В примере выше методы `say()` и `eat()`, помимо экземпляра класса, не принимали никаких аргументов. Однако методы являются функциями, поэтому мы без проблем можем расширить их функционал, например, добавив дополнительный параметр.

Приведенный ниже код:

```
class Cat:
    def say(self, sound):
        print(sound)

    def eat(self, meal):
        print(f'{meal} - это очень вкусно!')

cat1 = Cat()
cat2 = Cat()

cat1.say('Мяу')
cat1.eat('Молоко')

cat2.say('Мяяяу!')
cat2.eat('Рыба')
```

ВЫВОДИТ:

```
Мяу
Молоко - это очень вкусно!
Мяяяу!
Рыба - это очень вкусно!
```

Метод `__init__()`

Рассмотрим следующее определение класса `Cat` :

```
class Cat:
    pass
```

После создания экземпляра данного класса мы получаем объект, у которого нет никаких атрибутов. И если нам требуется, чтобы объект ими обладал, нам приходится определять их вручную для каждого объекта, либо определять атрибуты на уровне класса. Очевидно, оба способа имеют свои недостатки, так как в первом случае нам приходится определять каждый атрибут для каждого объекта, во втором случае — все экземпляры класса имеют атрибуты с одними и теми же значениями, что не всегда нужно. Решить данную проблему позволяет специальный метод `__init__()`.



Метод `__init__()` называют **методом инициализации** или **инициализатором**.

Метод `__init__()` инициализирует атрибуты объекта. Сразу после создания объекта выполняется метод `__init__()`, и параметру `self` автоматически присваивается объект, который был только что создан, что позволяет тут же наделить его необходимыми атрибутами.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

cat = Cat('Британский', 'Кемаль')
```

порода кошки
имя кошки

создает экземпляр класса `Cat`, который имеет атрибуты `breed` и `name` со значениями `Британский` и `Кемаль` соответственно.

1

В оперативной памяти создаётся объект на основе класса `Cat`

2

Вызывается метод `__init__()` класса `Cat`, и параметру **`self`** назначается вновь созданный объект

3

После выполнения этих шагов объект экземпляра класса `Cat` будет существовать с атрибутами **`breed`** со значением `“Британский”`, **`name`** со значением `“Кемаль”`

Экземпляр класса Cat

```
def __init__(self, breed, name):
    self.breed = breed
    self.name = name
```

Экземпляр класса Cat

`breed` → `“Британский”`
`name` → `“Кемаль”`

Важно уточнить, что метод `__init__()` выполняется после создания каждого экземпляра класса `Cat`, поэтому каждый объект будет иметь те значения атрибутов, которые были указаны при его создании.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

cat1 = Cat('Британский', 'Кемаль')
cat2 = Cat('Манчкин', 'Роджер')

print(cat1.breed, cat1.name)
print(cat2.breed, cat2.name)
```

ВЫВОДИТ:

Британский Кемаль
Манчкин Роджер

Поскольку мы определили параметры `breed` и `name` для метода `__init__()`, они должны быть явно переданы при создании новых экземпляров класса `Cat`.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

cat = Cat()
```

приводит к возбуждению исключения:

```
TypeError: Cat.__init__() missing 2 required positional arguments: 'breed' and 'name'
```

Однако если мы хотим, чтобы при создании объекта значения некоторых атрибутов можно было не указывать, мы можем установить им значения по умолчанию.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name=None):
        self.breed = breed
        self.name = name

cat1 = Cat('Британский', 'Кемаль')
cat2 = Cat('Манчкин')

print(cat1.breed, cat1.name)
print(cat2.breed, cat2.name)
```

ВЫВОДИТ:

Британский Кемаль
Манчкин None



Сигнатура метода `__init__()` полностью повторяет сигнатуру любой функции. С помощью `*args` и `**kwargs` метод `__init__()` может принимать произвольное количество позиционных и именованных аргументов соответственно.

Помимо значений аргументов, передаваемых в инициализатор, атрибутам можно устанавливать некоторые фиксированные значения.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name
        self.night_vision = True           # способность видеть в темноте

cat1 = Cat('Британский', 'Кемаль')
cat2 = Cat('Манчкин', 'Роджер')

print(cat1.night_vision)
print(cat2.night_vision)
```

ВЫВОДИТ:

```
True
True
```

Примечания

Примечание 1. Можно подумать, что метод `__init__()` отвечает за создание экземпляров класса, но это не так, он принимает в качестве аргумента уже созданный классом объект и инициализирует его.

Примечание 2. Возвращаемым значением метода `__init__()` должно быть значение `None`, в противном случае будет возбуждено исключение.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name
        return self

cat = Cat('Британский', 'Кемаль')
```

приводит к возбуждению исключения:

```
TypeError: __init__() should return None, not 'Cat'
```

Примечание 3. Класс может иметь только один инициализатор. При попытке определить в классе два инициализатора, предыдущий будет заменен следующим.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed

    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

cat = Cat('Британский', 'Кемаль')

print(cat.breed)
print(cat.name)
```

ВЫВОДИТ:

```
Британский
Кемаль
```

Примечание 4. Применение метода к объекту преобразуется в вызов функции, путем обращения к ней через соответствующий класс и передачи ей в качестве аргумента объекта, который этот метод вызвал.

Приведенный ниже код:

```
nums = [1, 2, 3]
text = 'beegeek'

nums.append(4)
text = text.lower()
```

эквивалентен следующему коду:

```
nums = [1, 2, 3]
text = 'beegeek'

list.append(nums, 4)
text = str.lower(text)
```

Это достаточно удобно использовать, если нам требуется воспользоваться методом как функцией.



